CS 331 - PROJECT T016 - PROJECT ID 13

Scalable Network I/O

Select → poll → epoll → io_uring

Four single-threaded TCP echo servers built from scratch and benchmarked from 10 to 1,000 concurrent connections — throughput, latency distributions, system-call frequency, and measured CPU/memory efficiency on a documented machine.

Kale Parth Hemant : select	Niraj Kumar : io_uring
Devvrat Hans : poll	Chirag Agarwalla : Benchmarking
Patil Nachiket Kiran : epoll	Nityanand Karmali : PPT and Report

- TA : Shivanshu

Problem Statement & Objectives

Problem statement and goal

Thread-per-connection servers stop scaling once a few thousand mostly-idle clients are open. This project implements select, poll, epoll and io_uring from scratch as single-threaded TCP echo servers, then benchmarks and profiles them under rising connection load — measuring, rather than asserting, where each mechanism starts to strain and why.

Deliverables

Four working C engines on port 9090, one per mechanism

Load-varied benchmarking at 10 / 100 / 1,000 connections

Syscall profiling, latency percentiles, CPU % and peak RSS

Written analysis of the O(N) vs O(ready) vs O(1) bottleneck

Questions that framed the presentation

Q1 How do throughput and latency change as concurrency rises from 10 to 100 to 1,000 connections?

Q2 How do event-wait and system-call frequencies change with load, and across mechanisms?

Q3 Why do select/poll carry O(N) scanning costs per call, while epoll scales with ready descriptors and io_uring batches?

Q4 Does io_uring's ring-buffer design measurably reduce user-to-kernel transitions versus epoll?

Q5 How do CPU utilization and memory footprint scale with connection count across the four engines?

Q6 What trade-offs remain even in the fastest mechanism, and where does this implementation still fall short?

Architecture / Mechanism

All four servers are single-threaded, listen on port 9090 and hold per-client state indexed by file descriptor. The loop is identical — only step 2 changes.

1 Accept
Connection arrives; per-client state created.

2 Discover
Ask the kernel which sockets are ready.

3 Read
Bytes pulled into a per-client buffer.

4 Echo
Same bytes written back; loop repeats.

select()

O(max_fd)

Two fd_set bitmaps rebuilt from the client list every iteration, because the call destroys them in place. The kernel walks every bit from 0 to the highest fd in use, active or not.

FD_SETSIZE = 1024 is a hard ceiling; guarded by an explicit fd >= FD_SETSIZE check and a rejection counter.

poll()

O(N) per call

A dynamically sized pollfd[] array replaces the bitmap and removes the ceiling, but kernel and application still walk the whole watched set on every call.

POLLOUT is armed only once a read leaves data unsent, so the write lands one iteration later.

epoll

O(ready)

An interest list created once with epoll_create1() lives in the kernel across calls. epoll_wait() returns only descriptors that are ready, so cost tracks active events.

One epoll_ctl() per state change; the read and its echo write happen in the same EPOLLIN dispatch.

io_uring

O(1) amortized

Whole operations are queued into a shared submission ring; the kernel runs them asynchronously and posts completions to a completion ring, drained in a batch per io_uring_enter().

Multishot accept armed once at startup; one outstanding op per client caps pipeline depth.

The scaling split: select and poll re-describe every watched descriptor on every call, so cost grows with connections watched, not connections active. epoll registers interest once and returns only the ready subset; io_uring removes the per-operation syscall boundary by batching through shared rings.

Extension / Issues Fixed / Evaluation

Five gaps from the first submission, now fixed

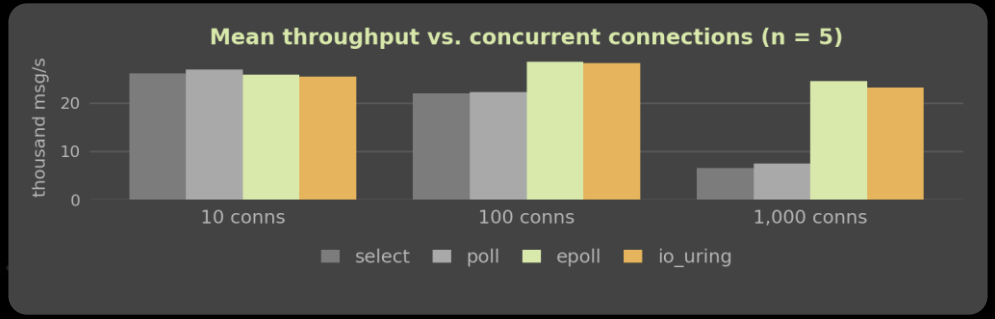
1. Unfair idle timeouts. select and poll blocked on finite timeouts (1 s / 1000 ms) while epoll and io_uring blocked indefinitely, inflating their syscall counts. Changed select()'s timeout argument &tv to NULL and poll()'s 1000 to -1, so all four now block infinitely when idle.

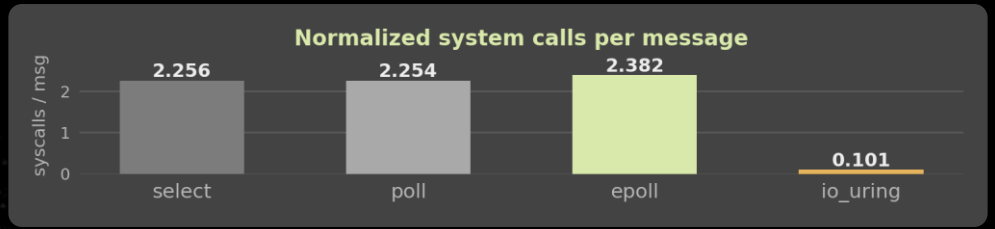
2. No CPU or memory data. Wrapped every server run in run_benchmark.sh with /usr/bin/time -v to capture Percent of CPU and Maximum resident set size for all 12 (engine x connection) configurations.

3. Truncated resource logs. The harness killed servers abruptly, so GNU time never flushed its summary. Re-ran all 12 configurations directly under /usr/bin/time -v and stopped each with SIGINT.

4. No true percentiles. The C client kept only a running mean/min/max, and tcpkali and wrk were unavailable here. Wrote latency_profiler.py, an asyncio client that times every round trip and sorts the sample array.

5. Undocumented machine. Captured lscpu, free -h, uname -r and gcc --version during the profiling session: Apple M2, 4 cores, 7.3 GiB RAM, Fedora kernel 7.1.6 aarch64, gcc 16.2.1.





Latency: at 1,000 connections mean latency is 605.8 / 533.2 µs for select/poll against 169.7 / 174.6 µs for epoll/io_uring. io_uring has the lowest p50 (7.8 ms), p95 and p99; epoll the worst tail (p99 37.6 ms). Event-wait calls per message: 0.50 against 0.25.

Yet epoll is marginally ahead on mean throughput, inside combined SD — the claim is architectural.

Non-Functional Testing Parameters

Performance

Throughput and mean latency timed with CLOCK_MONOTONIC over loopback; 64-byte request and reply, 100 messages per connection, 5 repetitions.

25.4k-26.9k msg/s at 10 conns; epoll and io_uring peak at 28.1k-28.4k at 100.

Scalability

The same workload swept across 10, 100 and 1,000 concurrent connections driven by 4 client worker processes, with plus/minus one SD error bars.

select and poll drop 70% / 66% from 100 to 1,000; epoll and io_uring stay flat.

Reliability

Every echoed reply compared byte-for-byte against what was sent; attempted and completed counts reconciled across all 60 core-benchmark runs.

Counts matched exactly. Zero content mismatches, all engines, all levels.

Resource efficiency

Each server wrapped in /usr/bin/time -v to read Percent of CPU this job got and Maximum resident set size at every connection level.

CPU 80% to 90%; RSS 1,040 KB and 1,072 KB for io_uring, load-independent.

System-call efficiency

strace -f -c traces normalized to syscalls per completed message, plus event-wait calls per message, after standardizing all engines to infinite idle blocking.

2.256 / 2.254 / 2.382 against 0.101 for io_uring; wait calls 0.50 vs 0.25.

Repeatability

Five repetitions per configuration with warm-up excluded, on one machine whose specification was captured during the profiling session itself.

Apple M2, 4 cores, 7.3 GiB, Fedora kernel 7.1.6 aarch64, gcc 16.2.1.

Scope: loopback only, so results isolate event-loop behaviour from network variability but say nothing about a real NIC, WAN link or packet loss. Profiling runs are 3-15x slower and were used only for syscall and resource counts, never mixed into throughput or latency figures.

Challenges Faced

1. No off-the-shelf load generator

tcpkali and wrk were unavailable on this Fedora aarch64 (Asahi) machine, so the whole measurement side had to be built in-house: the team's own C load generator for throughput, plus a separate Python client for latency.

2. Percentiles were unrecoverable

The C client only ever kept a running mean, min and max per run, so p50 and p99 could not be derived after the fact. It forced a second, purpose-built asyncio pass rather than a reprocessing of existing data.

3. Resource logs kept truncating

The benchmark harness terminated servers abruptly, so GNU time's summary never flushed. Each of the 12 configurations had to be re-run directly and stopped with SIGINT before clean CPU and RSS figures appeared.

4. Making the comparison genuinely fair

select and poll were idling on finite timeouts while epoll and io_uring blocked indefinitely — a confound that quietly inflated their syscall counts until every engine was standardized to infinite blocking.

5. select's architectural ceiling

FD_SETSIZE is fixed at 1024, so the 1,000-connection test point runs right against select's practical limit. The code needed an explicit fd >= FD_SETSIZE guard and a rejection counter rather than failing silently.

6. io_uring toolchain and depth cap

It needs kernel 5.19 or newer and liburing 2.2 or newer, links with the -luring flag, and our guard allowing one outstanding operation per client caps pipeline depth— which is why it does not beat epoll on raw throughput here.

Takeaway: most of the difficulty was in measurement fairness, not in writing the servers. The headline results only became defensible once these confounds were removed.